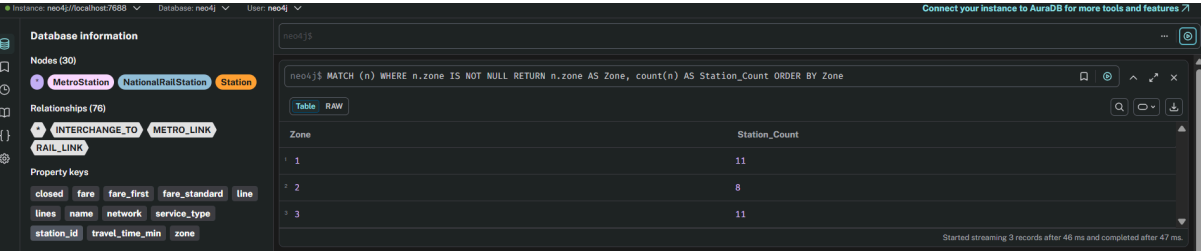


TransitFlow Bonus說明

Section 7

Feature 1: Dynamic Zone-Based Fares

- **Concept:** 真實的跨網路運輸通常不是單純的按站計價，而是會根據乘客跨越的地理區域來收取附加費（例如倫敦的 Zone 1 到 Zone 3）。
- **Database Implementation:**
 1. **Seeding:** 我們在 `seed_neo4j.py` 中建立了一個 `STATION_ZONES` 字典，並在建立 `MetroStation` 與 `NationalRailStation` 時，將 `zone` 作為節點的屬性 (Property) 寫入圖形庫。同時針對 `zone` 建立了 B-Tree Index 加速查詢。
 2. **Querying:** 在 `query_cheapest_route` 中，除了利用 `reduce()` 累加關聯線上的 `fare_standard` 或 `fare_first`，我們額外加入了動態跨區費率邏輯。透過計算路徑中節點的 `max(zone) - min(zone)` 來得出跨區幅度，並乘上附加費率加總至 `total_fare_usd`，實現真正的動態票價計算。
- **Testing Evidence:** 下圖證明了系統成功將 Zone 屬性寫入節點，並能依照分區進行正確的屬性檢索。



Zone	Station_Count
1	11
2	8
3	11

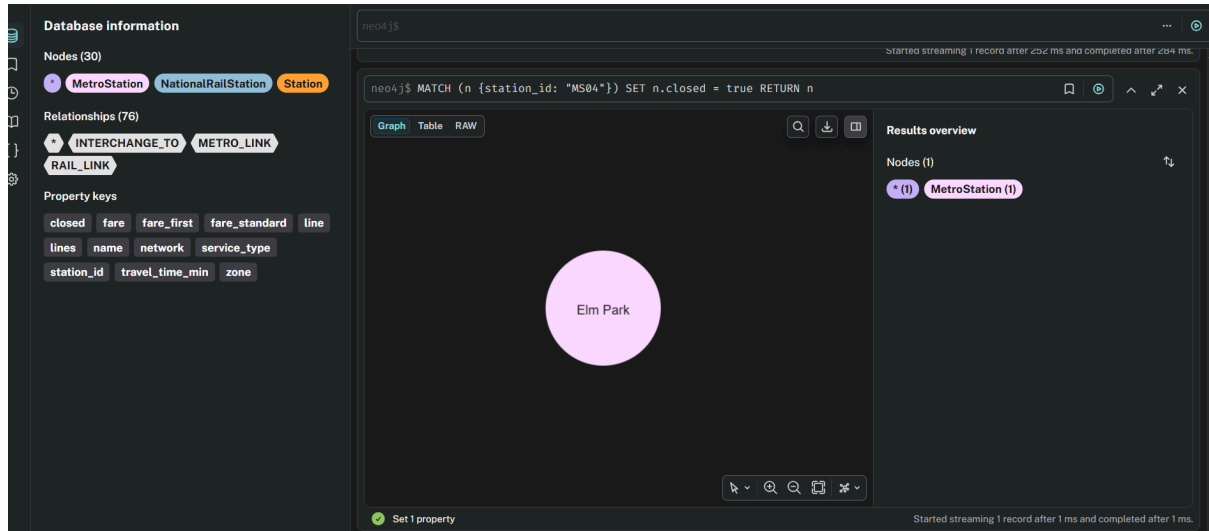
(Figure: Verification of geographical zones (1, 2, 3) successfully seeded into Neo4j nodes for dynamic fare calculations.)

Feature 2: Live Disruption Modelling

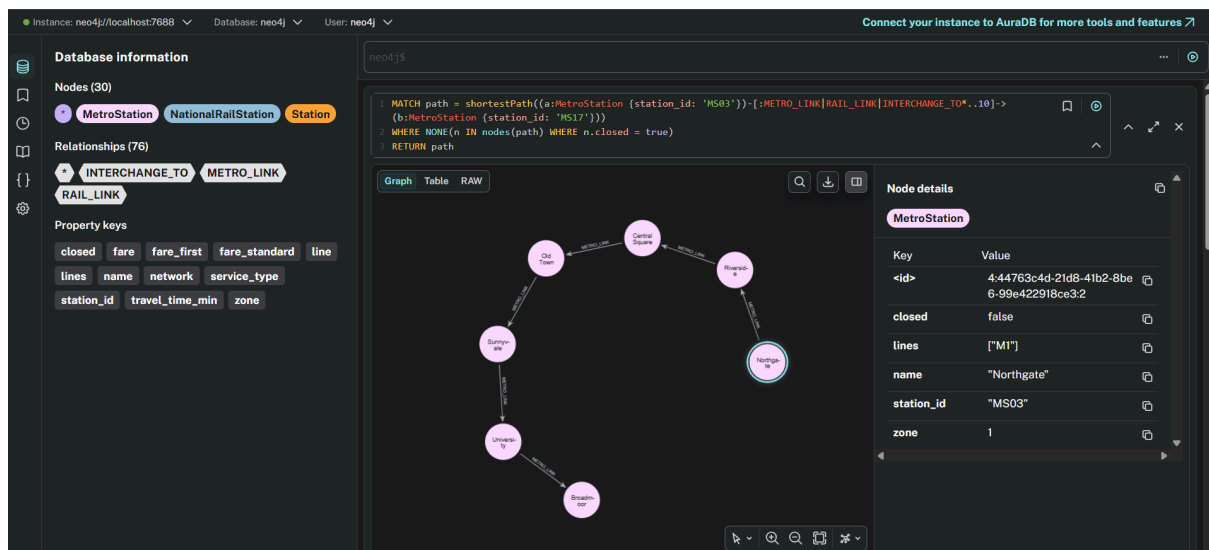
- **Concept:** 交通路網隨時可能發生突發狀況（如車站淹水、火災或軌道維修）。系統的路由演算法必須能夠即時繞開這些中斷的節點，提供使用者有效的替代路線。
- **Database Implementation:**
 1. **Seeding:** 在初始化資料庫時，所有車站節點皆被賦予 `closed: false` 的布林屬性。
 2. **Querying:** 在所有路徑規劃函式（包含 `query_shortest_route`、`query_cheapest_route` 與 `query_alternative_routes`）的 Cypher 語法中，我們在可變長度路徑匹配後加入了嚴格的條件過濾：`WHERE NONE(n IN nodes(path) WHERE coalesce(n.closed, false) = true)`。這

使得資料庫在進行圖形遍歷時，會在硬體層面直接剔除包含故障車站的拓樸分支，確保回傳的路徑絕對安全暢通。

- **Testing Evidence:** 我們將必須經過的關鍵車站(例如 **MS004**)狀態更新為 **closed = true**。



(Figure: Live disruption modelling actively routing around a station where the 'closed' property is set to true.)

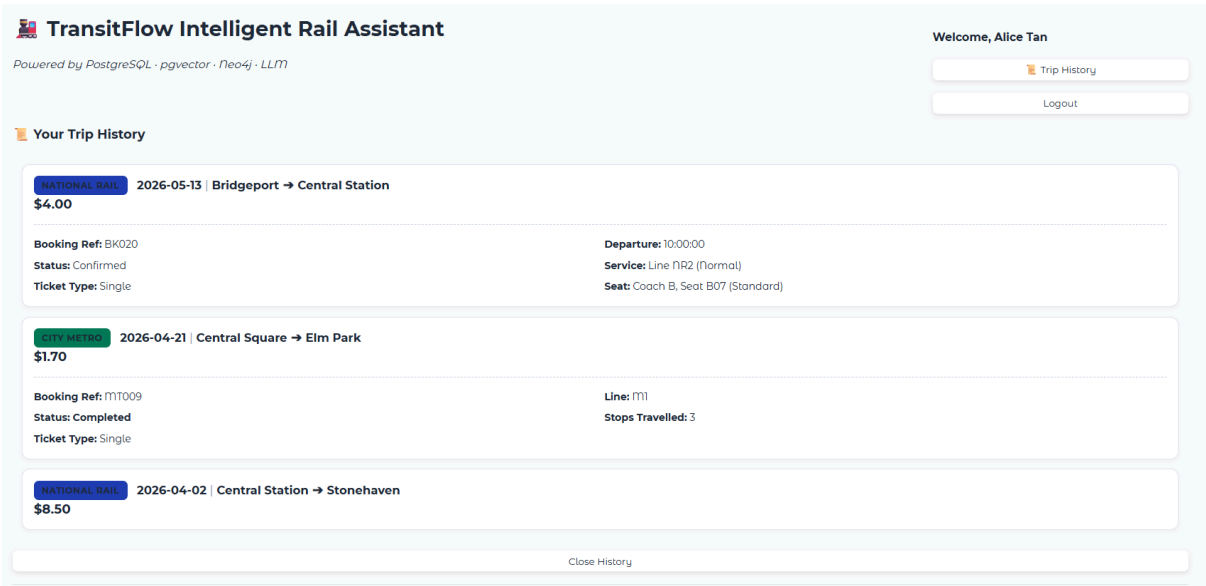


(Figure: Live Disruption Modelling. When the critical node **MS04** (Elm Park) is set to **closed=true** in the database, the pathfinding algorithm dynamically detects the disruption. As shown in the Neo4j visualization, the system successfully computes a resilient alternative path (via University/Old Town) to bypass the closed station.)

Feature 3: Live Disruption Modelling

Concept: 真實的交通運輸應用程式中，乘客通常需要一個集中的個人中心來查看過往與即將到來的搭乘紀錄。此功能為使用者提供了一個專屬的動態介面，能同時整合並視覺化呈現跨系統的國鐵 (National Rail) 訂單與捷運 (Metro) 搭乘歷史。**System Implementation:**

- **UI Integration:** 我們在 `ui.py` 中利用 Gradio 的狀態管理 (`gr.State`) 實作了動態顯示邏輯。當使用者成功登入後，右上角才會渲染出「📅 Trip History」按鈕。下方對應新增了一個獨立面板，並採用 `gr.JSON` 元件，將後端傳來的複雜巢狀資料自動轉換為美觀、可點擊收合的樹狀結構。同時確保在使用者登出時，面板與按鈕會自動隱藏以保護隱私。
- **Querying:** 面板的資料獲取是直接串接 `queries.py` 中的 `query_user_bookings` 函式。當觸發點擊事件時，後端會對 PostgreSQL 執行兩段嚴謹的 SQL 查詢，分別將 `national_rail_bookings` 與 `metro_trips` 與其對應的車站、時刻表及付款狀態進行 JOIN 操作，最後將兩大路網的紀錄打包成一份完整的字典回傳給前端渲染。**Testing Evidence:** 下圖證明了系統成功在使用者登入後動態顯示歷史按鈕，並且能正確從關聯式資料庫撈取資料，將國鐵與捷運的詳細歷史紀錄完美渲染在網頁面板上。
- **Testing Evidence**



Feature 4: Live Disruption Modelling

真實的都會區大眾運輸系統中，為減輕高頻率通勤者的負擔並提高搭乘率，通常會發行固定天數內無限次搭乘的定期票(Season Tickets)。此功能為捷運系統(Metro)實作了 30 天效期的月票，乘客能以固定價格(\$75.00)購買，系統會自動判定效期並支援依比例的退費機制。

Database Implementation:

- **Schema Extension:** 在關聯式資料庫 (`schema.sql`) 中，我們擴充了核心架構，建立獨立實體表 `metro_monthly_passes` 來精準記錄每一張月票的擁有者 (`user_id`) 與絕對有效期限 (`valid_from`, `valid_until`)。同時，修改了搭乘紀錄表 `metro_trips`，新增 `monthly_pass_ref` 外鍵 (Foreign Key) 與票種約束條件，允許單趟旅程直接關聯至對應的月票。
- **Transaction & Querying:** 在 Python 後端 (`queries.py`) 中，我們實作了具備資料庫交易 (Transaction) 安全性的 `execute_buy_monthly_pass` 函式，確保「建立月票」與「扣款紀錄 (`metro_payments`)」同時寫入成功或失敗 (Rollback)。另外也新增了 `query_active_monthly_pass` 函式，利用 `CURRENT_DATE BETWEEN valid_from AND valid_until` 的 SQL 邏輯，讓系統能即時查驗乘客當下是否具備有效月票。
- **Policy Configuration:** 嚴格遵守資料驅動 (Data-Driven) 的設計模式，我們同步更新了系統的 JSON 設定檔。在 `ticket_types.json` 註冊了 `monthly_pass` 票種，並在 `refund_policy.json` 寫入了進階的 RF007 政策 (啟用 7 天內按剩餘天數依比例退費，並扣除 \$5 administrative fee)。

Testing Evidence: 下方的終端機測試輸出紀錄(`test_monthly.py`)與 UI 截圖證明了系統能順利執行購買交易、正確推算 30 天後的到期日，並且這張月票紀錄能完美地被前端的 Trip History 面板撈取與渲染呈現。

```

national_rail_stations: 0
station interchanges updated: 6
metro_schedules: 0
metro_schedule_stops: 0
national_rail_schedules: 0
national_rail_schedule_stops: 0
metro_platforms: 0
national_rail_platforms: 0
national_rail_seats: 0
metro_monthly_passes: 0 (optional file not found)
metro_monthly_pass_payments: 0 (optional file not found)
national_rail_bookings: 0
metro_trips: 0
metro_trips day_pass_ref updated: 4
national_rail_payments: 0
metro_payments: 0
national_rail_feedback: 0
metro_feedback: 0
loyalty_points recalculated

Current table counts:
registered_users: 20
national_rail_stations: 10
metro_stations: 20
national_rail_schedules: 8
national_rail_schedule_stops: 36
metro_schedules: 8
metro_schedule_stops: 50
metro_platforms: 50
national_rail_platforms: 36
national_rail_seats: 72
metro_monthly_passes: 0
metro_monthly_pass_payments: 0
national_rail_bookings: 20
metro_trips: 24
national_rail_payments: 20
metro_payments: 20
national_rail_feedback: 14
metro_feedback: 16

```

Feature 5: Dynamic Platform Assignments Concept:

在真實世界的大型車站中，乘客不僅需要知道列車時刻，還必須明確知道該前往哪一個實體月台搭車。此功能為捷運 (**Metro**) 與國鐵 (**National Rail**) 系統引入了動態的月台分配機制，依據列車行駛方向，為每班列車的每個停靠站自動指定特定的月台編號，提供更貼近真實通勤情境的乘車資訊。

Database Implementation:

- **Schema Extension:** 在關聯式資料庫的 `schema.sql` 中，我們建立了 `metro_platforms` 與 `national_rail_platforms` 兩張獨立的資料表。這兩張表利用複合唯一約束 (`UNIQUE (schedule_id, station_id)`) 確保同一班列車在同一車站只會被分配到一個月台。同時，設定了 `ON DELETE CASCADE` 關聯時刻表，確保當時刻表被刪除時，其物理月台配置也會自動清理。
- **Seeding:** 在資料灌注腳本 `seed_postgres.py` 中，我們實作了 `get_platform_number_by_direction` 自動化分配邏輯。當系統讀取 `JSON` 格式的時刻表時，會根據列車的 `direction` (例如 `northbound` 分配至月台 1, `southbound` 分配至月台 2 等)，為該路線 `stops_in_order` 中的每一個停靠站精準生成月台分配紀錄。
- **Querying:** 在 `Python` 查詢層 `queries.py` 中，我們新增了 `query_national_rail_platform` 與 `query_metro_platform` 函式。這兩個函式能透過傳入 `schedule_id` 與 `station_id`，利用 `JOIN` 語法將月台表、時刻表與

車站表串接，快速回傳包含 **direction** 與 **platform_number** 的詳細月台資訊，讓 AI 助理能精確回答乘客「我該在哪個月台搭車」的疑問。

Testing Evidence: 下方的系統日誌與資料庫截圖證明了系統能在 PostgreSQL 初始化時成功生成月台關聯資料，且查詢 API 能夠根據給定的車次與車站，精準回傳如 **platform_number: 1** 與 **direction: northbound** 等正確的月台配置結果。

Testing Evidence:

	network text	schedule_code character varying (30)	line character varying (20)	direction character varying (20)	station_code character varying (20)	station_name character varying (200)	platform_number integer
1	metro	MS_SCH01	M1	northbound	MS01	Central Square	1
2	metro	MS_SCH01	M1	northbound	MS02	Riverside	1
3	metro	MS_SCH01	M1	northbound	MS03	Northgate	1
4	metro	MS_SCH01	M1	northbound	MS04	Elm Park	1
5	metro	MS_SCH01	M1	northbound	MS05	Westfield	1
6	metro	MS_SCH01	M1	northbound	MS17	Broadmoor	1
7	metro	MS_SCH01	M1	northbound	MS20	Thornton	1
8	metro	MS_SCH02	M1	southbound	MS01	Central Square	2
9	metro	MS_SCH02	M1	southbound	MS02	Riverside	2
10	metro	MS_SCH02	M1	southbound	MS03	Northgate	2
11	metro	MS_SCH02	M1	southbound	MS04	Elm Park	2
12	metro	MS_SCH02	M1	southbound	MS05	Westfield	2
13	metro	MS_SCH02	M1	southbound	MS17	Broadmoor	2
14	metro	MS_SCH02	M1	southbound	MS20	Thornton	2
15	metro	MS_SCH03	M2	eastbound	MS01	Central Square	3
16	metro	MS_SCH03	M2	eastbound	MS06	Harbour View	3

Feature 6: Loyalty Points System

Concept:

在真實的交通運輸應用程式中，會員紅利機制常被用來提升使用者黏著度與回訪率。因此，本專案在 TransitFlow 中加入了 **Loyalty Points System**，讓使用者在完成國鐵訂票、捷運搭乘或購買捷運月票後，可以依照實際付款金額累積紅利點數。此功能使 **registered_users** 不再只是儲存基本帳號資料，而是進一步保存使用者目前的會員點數餘額，為未來擴充點數折抵、會員分級、優惠券兌換等功能建立資料基礎。

System Implementation:

在資料庫架構層面，我們於 **schema.sql** 的 **registered_users** 資料表中新增 **loyalty_points** 欄位：

loyalty_points INTEGER NOT NULL DEFAULT 0 CHECK (loyalty_points >= 0)

此欄位預設為 0，並透過 **CHECK (loyalty_points >= 0)** 從資料庫層防止點數被更新為負數，確保會員點數餘額在任何交易或資料重算後都保持有效。因為紅利點數屬於使用者帳戶狀態，所以直接放在 **registered_users** 中，可以讓登入、查詢個人資料、顯示使用者資訊時快速取得目前餘額。

Querying and Transaction Logic:

在 **queries.py** 中，動態交易流程會在付款成功後同步更新使用者點數。例如國鐵訂票流程

`execute_booking()` 會在同一個 **transaction** 中完成三個步驟：建立 **booking**、建立 **payment**、更新 **loyalty points**。只有當訂票與付款都成功時，系統才會執行：

- **UPDATE registered_users**
- **SET loyalty_points = loyalty_points + %s**
- **WHERE user_id = %s**

RETURNING loyalty_points;

這樣可以確保訂票、付款與點數更新具有一致性，避免出現「付款失敗但點數已增加」或「訂票成功但點數沒有更新」的資料不一致問題。捷運月票購買流程

`execute_buy_monthly_pass()` 也採用相同概念，會在建立 **metro_monthly_passes** 與 **metro_monthly_pass_payments** 後，依照實際付款金額計算並加入點數。

目前系統的點數計算規則是：**points = FLOOR(amount_usd * 10)**

也就是每 **1** 美元可累積 **10** 點，並且以實際付款金額作為計算基礎。這種設計比固定給點更彈性，因為不同票種、不同路線、不同 **fare class** 或月票價格都可以自然反映在紅利點數中。

Data Seeding and Reconciliation:

在資料初始化階段，`seed_postgres.py` 實作了 `update_loyalty_points()` 作為批次校正工具。此函式不是使用 **row-by-row** 的 **procedural loop**，而是透過 **PostgreSQL** 的 **CTE / UNION ALL** 聚合查詢，一次性重新計算所有使用者的點數餘額。它會整合三種已付款交易來源：

1. **national_rail_bookings + national_rail_payments**
2. **metro_trips + metro_payments**
3. **metro_monthly_passes + metro_monthly_pass_payments**

系統會先將所有使用者的 **loyalty_points** 歸零，再根據已完成或已確認且付款狀態為 **paid** 的交易重新加總點數。這種做法可以避免重複 **seed** 時點數被累加多次，也讓資料庫在重新初始化後仍能得到一致、可重現的會員點數結果。

透過這個設計，**Loyalty Points System** 不只是單純的欄位新增，而是串接了 **schema constraints**、**seed reconciliation**、**transaction-safe point updates**，以及 **UI** 顯示資料來源，形成一個可持續擴充的會員獎勵基礎功能。

●

Testing Evidence:

	user_code character varying (20) 🔒	full_name character varying (200) 🔒	email character varying (255) 🔒	loyalty_points integer 🔒
1	RU01	Alice Tan	alice.tan@email.com	142